計算物理学B

第四回：配列（NumPy）

野垣 康介、藤本 悠輝

講義予定

2026 年度 第 2 学期（火曜 3 限）

10/6	四則演算	12/8	モンテカルロ 1
10/27	制御文（for, if）	12/15	モンテカルロ 2
11/6（金）	関数	12/22	微分方程式 1
11/10	**配列（NumPy）**	1/12	微分方程式 2
11/17	可視化（matplotlib）	1/19	偏微分方程式 1
11/24	数値微分	1/26	偏微分方程式 2
12/1	数値積分	2/2	根の探索
	～中間レポート～		～期末レポート～

10/20 は出張で休講。

11/6（金）は火曜授業の振替。

日付：令和 8 年度（2026）新潟大学授業暦

授業で使うリンク

- Google Colab
  https://colab.research.google.com/?hl=ja
- 講義資料（GitHub）
  https://github.com/nogaki/Computational_Physics_B
- 今週の教材：`2026/week4/week4.ipynb`

GitHub から Colab へ取り込む

1. GitHub で `week4.ipynb` を開き、
   ブラウザーのアドレス欄の URL をコピーします。

2. Colab で「ファイル」→「ノートブックを開く」
   →「GitHub」を選びます。

3. コピーした URL を貼り付けて検索し、表示されたノートブックを開きます。

開いたら「ドライブにコピーを保存」

「ファイル」メニューから、自分用のコピーを保存します。
開いただけでは、自分のドライブに保存されません。

物理量を時刻ごとに並べる

空気抵抗を無視し、初速 0、落下開始位置 $x = 0$、下向きを正として自由落下を考えます。

$g = 10.0\ \mathrm{m/s^2}$ と近似すると、落下開始からの経過時間 $t \geq 0$ で $x = \frac{1}{2}gt^2$、$v = gt$ です。

時刻 $t\,[\mathrm{s}]$	0	1	2	3
位置 $x\,[\mathrm{m}]$	0	5	20	45
速度 $v\,[\mathrm{m/s}]$	0	10	20	30

各列は同じ時刻の物理量です。時刻の点数を増やしても、この対応を保って計算します。

時刻と位置をリストにする

時刻と位置を同じ順番のリストに並べます。times の単位は秒、positions はメートルです。

```
times = [0.0, 1.0, 2.0]
positions = [0.0, 5.0, 20.0]
print(times[1], positions[1])
print(len(times))
```

出力
```
1.0 5.0
```

出力
```
3
```

同じ添字が同じ時刻を表します。添字は 0 から始まり、ここでは times[1] と positions[1] が 1 秒と 5 メートルです。

計算結果を追加して取り出す

3 秒後の位置 $45.0\ \mathrm{m}$ も、時刻と位置の両方に追加します。

```
times.append(3.0)
positions.append(45.0)
for i in range(len(times)):
    print(times[i], positions[i])
```

出力（各行）

```
0.0 0.0
1.0 5.0
2.0 20.0
3.0 45.0
```

リスト名の後に付ける操作

- `.append(x)`：末尾に x を追加
- `.pop()`：末尾を削除して値を返す
- `.pop(i)`：添字 i の要素を削除して返す
- `.remove(x)`：最初に一致する x を削除

時刻と位置の両リストから、同じ添字の要素を削除します。

前回の関数を使う

前回の free_fall で、3 秒後の位置と速度を求めます。

```
def free_fall(t):
    g = 10.0
    x = 0.5 * g * t ** 2
    v = g * t
    return x, v

x_one, v_one = free_fall(3.0)
print(x_one, v_one)
```

出力 `45.0 30.0`

出力は位置（m）、速度（m/s）の順です。

array と ndarray の違い

np.array は配列を作る関数、ndarray はその型です。

```
import numpy as np
t = np.array([0.0, 1.0, 2.0, 3.0])
print(type(t))
```

出力 `<class 'numpy.ndarray'>`

`list`	[] で作る組込みのリスト。型を混在でき、* 2 は繰り返し。
`array.array`	Python 標準ライブラリの 1 次元配列。型を揃えて保存し、* 2 は繰り返し。
`numpy.ndarray`	1 次元も含む多次元配列。要素型（dtype）を揃え、* 2 は各要素を 2 倍。

配列で全時刻を計算する

この関数の*と**は、配列の各要素に作用します。

```
x, v = free_fall(t)
print(x)
print(v)
```

出力

```
x  [0. 5. 20. 45.]
v  [0. 10. 20. 30.]
```

時刻の点数を増やすときも、同じ関数を使えます。

```
t = np.linspace(0.0, 3.0, 7)
x, v = free_fall(t)
```

0 秒と 3 秒を含む 7 点、間隔 0.5 秒です。

x[i] と v[i] は、同じ時刻 t[i] に対応します。

同じ時刻のデータを取り出す

```
print(t[2], x[2], v[2])
print(t[:3])
print(x[:3])
```

出力
```
1.0  5.0  10.0
```
出力
```
[0. 0.5 1.]
```
出力
```
[0. 1.25 5.]
```

添字 2 は 1 秒で、位置 5 メートル・速度 10 メートル毎秒です。

[:3] は添字 0,1,2 を取り出し、終点 3 は含みません。

各時刻の運動エネルギー

質量 $m = 2.0\,\mathrm{kg}$ の物体の運動エネルギーは $E = \frac{1}{2}mv^2$ です。

```
mass = 2.0
energy = 0.5 * mass * v ** 2
print(energy)
```

出力　`[0. 25. 100. 225. 400. 625. 900.]`

`energy[i]` は `t[i]` での運動エネルギー（単位 J）です。静止からの自由落下では、失った位置エネルギー mgx に等しくなります。

多次元配列：行と列

1 次元は添字 1 つ、2 次元は行・列の 2 つで要素を指定します。

```
r = np.array([
    [1, 0],
    [0, 2],
    [-1, 0]
])
```

行は粒子、列は座標（単位 m）。

	列 0（x）	列 1（y）
行 0	1	0
行 1	0	2
行 2	-1	0

`r.shape` → `(3, 2)`
3 行・2 列の 2 次元配列です。

`r[1, 0]` `0` 行 1・列 0 の要素
`r[1]` `[0 2]` 行 1 の全体（2 番目の粒子）
`r[:, 0]` `[1 0 -1]` 全行の列 0（`:` は全て）

時刻ごとにこの表を重ねると、（時刻数, 3, 2）の 3 次元配列になります。

行列で全粒子を回転する

粒子の位置を原点のまわりに反時計回り 90 度回転します。

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = R \begin{pmatrix} x \\ y \end{pmatrix}, \qquad R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

行データには右から R^T を掛けます。@は行列積、.T は転置、*は要素積です。

```
R = np.array([[0, -1], [1, 0]])
rotated = r @ R.T
print(rotated)
```

出力 $\begin{bmatrix} 0 & 1 \\ -2 & 0 \\ 0 & -1 \end{bmatrix}$

繰り返し測った周期のばらつき

仮想の振り子周期 4 件です（実測値ではなく、単位 s）。

```
periods = np.array([1.75, 2.0,
                    2.25, 2.0])
print(np.mean(periods))
print(np.var(periods))
```

出力：平均 $2.0\ \mathrm{s}$、分散 $0.03125\ \mathrm{s}^2$。測定値 T_i（$i = 1, \ldots, n$）、測定回数 n：$\bar{T} = \frac{1}{n}\sum_{i=1}^{n} T_i$　　$\sigma_T^2 = \frac{1}{n}\sum_{i=1}^{n}(T_i - \bar{T})^2$

`np.var` の既定分母は n。分散は測定値の散らばりで、平均値の誤差とは別です。

仮想的な周期測定データを作る

真の周期は $T_0 = 2.0\ \mathrm{s}$。誤差は独立な正規分布（平均 0、標準偏差 $0.1\ \mathrm{s}$）とする仮想モデルです。

```
rng = np.random.default_rng(0)
samples = rng.standard_normal(1000)
samples = 2.0 + 0.1 * samples
values = samples.tolist()
```

1000 回分の仮想測定です。理論平均 $2.0\ \mathrm{s}$、分散 $0.01\ \mathrm{s}^2$ ですが有限標本ではずれます。

`values` は同じデータのリスト。seed を固定すると同じ環境で再現しやすくなります。

物理の例で配列を使う

1. 0〜3 秒の 31 点で、自由落下の位置・速度と、質量 $2.0\ \mathrm{kg}$ の運動エネルギーを配列で求めましょう。

2. 「行列で全粒子を回転する」（本文 13 ページ）の例で使った 3 粒子（回転前の座標は $(1, 0)$, $(0, 2)$, $(-1, 0)$）を考えます。回転前の `r` と 90 度回転後の `rotated` を使い、原点からの距離の 2 乗 $x^2 + y^2$ が粒子ごとに等しいか確かめましょう。

   ヒントは `r[:, 0]**2 + r[:, 1]**2` です。

3. `samples` の平均と分散を求め、理論値の $2.0\ \mathrm{s}$、$0.01\ \mathrm{s}^2$ と比べましょう。

`for` 文による統計計算と実行時間の比較は、ノートブックの補足・発展課題にあります。

関数値を配列に保存して使う

$f(x) = \exp[-(x-0.3)^2]$ を格子点で評価します。
座標と関数値は同じ添字で対応します。

4. $[-1, 1]$ の両端を含む等間隔の 21 点を `x_grid` に入れます。Python で関数 `f` を定義し、各点での関数値を `f_grid` に保存しましょう。両配列の形状と先頭の値を表示しましょう。

5. 両配列を `[::4]` で間引きます。元の 21 点と間引いた 6 点で、関数値が最大となる点の座標と関数値を配列から求めて比べましょう。`np.argmax` は最大値の添字を返します。

計算物理学B

第四回付録：array と ndarray

野垣 康介、藤本 悠輝

３種類の配列：名前と役割

名前	できること・特徴
`list`	Python 組込み。異なる型を混ぜられ、要素を追加・削除できます。
`array.array`	標準ライブラリの可変１次元配列。型コードで要素型をそろえます。
`numpy.ndarray`	NumPy の１次元・多次元配列。dtype と shape を持ち、要素ごとの計算や行列計算に便利です。

`np.array(...)` は作成関数、`ndarray` は配列の型です。１次元の結果も `ndarray` です。

作り方と型の指定

```
import array as ar
import numpy as np
a_std = ar.array("d", [1.0, 2.0])
a_np = np.array([1.0, 2.0],
   dtype=float)
print(a_std.typecode, a_np.dtype,
   a_np.shape)
```

出力

```
d float64 (2,)
```

'd' は C の double、'i' は C の int（幅は環境依存）で、型コードは Python の版にも依存します。np.array(...) は型を推定し、dtype で明示もできます。typecode とは別の指定方式です。

同じ演算子でも意味が変わる

```
a_std = ar.array("d", [1.0, 2.0])
a_np = np.array([1.0, 2.0])
print((a_std * 2).tolist())
print(a_np * 2)
```

出力

```
[1.0, 2.0, 1.0, 2.0]
[2. 4.]
```

同じ型コードの標準 array の+は連結、* 2 は繰り返し。**は TypeError です。
ndarray の+、*、**は要素ごとの演算、@は行列積です。

標準 array の追加・削除

すべて元配列を書き換えます。

append(x)・extend(xs)：末尾へ追加。
insert(i, x)：添字位置へ追加。
del a[i]：その要素を削除。
pop(i)：削除して値を返す。i 省略時は末尾。
remove(x)：最初に一致する値を 1 つ削除。

```
a = ar.array("d",
    [1, 2, 2, 3])
a.append(4)
print(a.pop())
a.remove(2)
print(a.tolist())
```

出力

```
4.0
[1.0, 2.0, 3.0]
```

範囲外・空の pop は IndexError、見つからない値への remove は ValueError です。

NumPyの追加は返り値を受け取る

ndarrayにappendメソッドはありません。np.append・np.insertは新配列を返します。

```
a_np = np.array([1.0, 2.0, 2.0, 3.0])
grown_np = np.append(a_np, 4)
inserted_np = np.insert(a_np, 1, 9)
print(a_np, grown_np, inserted_np,
   sep="\n")
```

出力

```
[1. 2. 2. 3.]
[1. 2. 2. 3. 4.]
[1. 9. 2. 2. 3.]
```

元に反映するには返り値を代入します。

NumPyの削除：値か、添字か

np.deleteは添字を指定し、条件式は条件に合う値を選びます。

```
a_np = np.array([1.0, 2.0, 2.0, 3.0])
print(np.delete(a_np, 1))
print(a_np[a_np != 2])
print(a_np)
```

出力

```
    [1. 2. 3.]
    [1. 3.]
    [1. 2. 2. 3.]
```

np.delete(a_np, 1)は添字１を削除。a_np[a_np != 2]は値２をすべて除きます。どちらも新配列を返します。直接の添字代入は元を更新します。

多次元配列の追加と axis

```
m = np.array([[1, 0], [0, 2]])
r = np.array([[-1, 0]])
joined = np.concatenate((m, r), axis=0)
flat = np.append(m, r)
print(joined.shape, flat.shape)
```

出力

```
(3, 2) (6,)
```

axis=0 は行、axis=1 は列。結合する軸以外の大きさをそろえます。

np.append・np.insert・np.delete は既定の axis=None で平坦化。np.concatenate は既定の axis=0 です。

スライスはコピーか、共有か

```
a_std = ar.array("d", [1, 2, 3, 4])
s_std = a_std[1:3]
s_std[0] = 99
a_np = np.array([1., 2., 3., 4.])
s_np = a_np[1:3]
s_np[0] = 99
print(a_std.tolist(), a_np, sep="\n")
```

出力

```
[1.0, 2.0, 3.0, 4.0]
[ 1. 99.  3.  4.]
```

view と copy・高度な添字

```
a_np = np.array([1., 2., 3., 4.])
selected_np = a_np[[1, 2]]
selected_np[0] = 99
copy_np = a_np[1:3].copy()
copy_np[0] = -1
print(a_np)
```

出力

```
[1. 2. 3. 4.]
```

標準 array のスライスはコピー、NumPy の基本スライスは view です。整数配列・条件配列による選択結果と .copy() も独立配列です。直接の添字代入は元を更新し、b = a は同じオブジェクトを参照します。

shape・reshape・転置・行列積

```
matrix_np = np.arange(4).reshape(2, 2)
vector_np = np.array([1, 2])
print((matrix_np @ vector_np).tolist())
print(matrix_np.T.tolist())
print(vector_np.shape,
   vector_np.T.shape)
```

出力

```
    [2, 8]
    [[0, 2], [1, 3]]
    (2,) (2,)
```

reshape は要素数を保ち、結果は view または copy です。@は行列積。.T は２次元では転置、１次元では形を変えません。

整数型への小数代入：標準 array

array('i') の要素は C の int 型です。小数はそのまま代入できません。

```
int_std = ar.array("i", [1, 2])
int_std[0] = -2.5
```

出力

```
TypeError
```

整数ndarrayへ小数を代入すると

```
int_np = np.array([1, 2], dtype=int)
int_np[0] = -2.5
float_np = np.array([1, 2],
                    dtype=float)
float_np[0] = -2.5
print(int_np, float_np)
```

出力

```
[-2  2] [-2.5  2. ]
```

この代入では小数部が0方向に落ち、-2になります。小数を保持するならdtype=floatで作ります。これはこの要素代入例の結果です。

相互変換とコピー

```
source_std = ar.array("d", [0, 1])
converted_np = np.array(source_std)
converted_np[0] = 99
print(source_std.tolist(),
   converted_np)
print(np.sin(source_std),
   type(np.sin(source_std)))
```

出力

```
    [0.0, 1.0] [99. 1.]
    [0. 0.84147098] <class 'numpy.ndarray'>
```

np.sin は標準 array を受け取り ndarray を返します。共有・コピー条件は変換方法で異なります。

2次元配列を標準arrayへ戻す

1次元なら ar.array('d', nd.tolist()) で戻せます。2次元の tolist() は入れ子なので、平坦化します。

```
matrix_np = np.array([[1.0, 2.0],
   [3.0, 4.0]])
flat_std = ar.array("d",
   matrix_np.flatten().tolist())
print(flat_std.tolist())
```

出力

```
    [1.0, 2.0, 3.0, 4.0]
```

flatten() 後は値が並ぶだけで、(2, 2) の形は残りません。形が必要なら ndarray で扱います。

自由落下を標準arrayで計算する

静止・原点から落下、下向きを正、空気抵抗なし、$g = 10.0\ \mathrm{m/s^2}$ とします。$x(t) = \frac{1}{2}gt^2 = 5.0t^2 \quad (t \geq 0)$

```
times_std = ar.array("d", [0., 1., 2.])
positions_std = ar.array("d")
for time_value in times_std:
    positions_std.append(
        5.0 * time_value ** 2)
print(positions_std.tolist())
```

出力

```
[0.0, 5.0, 20.0]
```

時刻を順番に走査し、各時刻の位置を append で追加します。

同じ自由落下を ndarray で計算する

同じ条件・時刻では、配列演算で全位置をまとめて求められます。

```
times_np = np.array([0., 1., 2.])
positions_np = 5.0 * times_np ** 2
print(positions_np)
```

出力

```
[ 0.  5. 20.]
```

1 次元の同型値を標準ライブラリだけで扱うなら array.array、一括計算・多次元・行列計算なら ndarray が便利です。速度は実際の条件で測ります。

仕様・使用例：Python：array、NumPy：基本操作、コピーと view

np.append、np.delete、np.array